

FPGA Edge-IIoT Intrusion Detection: Hierarchical Deep Learning Implementation

Abstract

We propose a multi-stage Intrusion Detection System (IDS) for Industrial IoT (IIoT) security that combines a Linear Regression filter, a Convolutional Autoencoder, and a Convolutional Neural Network (CNN) with Spatial Attention within a conditional inference pipeline. Each module is implemented on an Xilinx PYNQ-Z2 FPGA (Zynq-7000 SoC). A custom Verilog IP performs Linear Regression inference, while the quantized CNN and Convolutional Autoencoder models are deployed using the Xilinx FINN library. Spatial Attention is employed in the CNN because it provides spatially aware feature weighting that maps efficiently to FPGA dataflow architectures. The hierarchical cascade invokes the computationally expensive CNN only for suspicious samples, achieving ~90% pipeline-level detection accuracy (binary: attack vs. normal) while the CNN produces 15-class attack-type labels and is triggered on fewer than 2% of inputs. We detail preprocessing of the Edge-IIoTset dataset, model architectures, quantization strategies, and the FINN compilation workflow, and present comprehensive analyses including training curves, ROC characteristics, threshold evaluation, confusion matrices, and hardware resource utilization (LUT, FF, BRAM, DSP), latency, and throughput.

1. Introduction

The rapid growth of Industrial IoT (IIoT) deployments has introduced significant security concerns. Anomaly-based Intrusion Detection Systems (IDS) analyse network traffic to identify potential attacks, but deep learning models often incur high computational costs. Continuously running a resource-intensive CNN at the edge leads to inefficient utilisation of FPGA hardware.

To address this, we propose a hierarchical IDS architecture in which lightweight models operate continuously and invoke a more complex model only when suspicious activity is detected. Specifically, a Verilog-based Linear Regression (LR) IP performs high-speed initial filtering, followed by a quantized Convolutional Autoencoder (Conv-AE) for anomaly detection, and finally a CNN with Spatial Attention for accurate threat confirmation. Spatial Attention is selected because it concentrates on spatially salient feature regions and produces compact, hardware-friendly attention maps well-suited for FPGA dataflow pipelines.

Unlike conventional single-model FPGA IDS, we propose a dynamically gated, multi-stage inference architecture with conditional execution at the hardware level. Early-stage lightweight models filter the majority of benign traffic, allowing only a small subset of suspicious inputs to activate the computationally intensive CNN. This achieves a favourable trade-off between detection performance, latency, and energy consumption, making the system well-suited for real-time IIoT edge deployments.

2. Related Work

2.1 FPGA-based Intrusion Detection

FPGA accelerators have been extensively explored for high-speed Network Intrusion Detection Systems (NIDS), primarily due to their ability to process network traffic in parallel and sustain high

throughput. Early FPGA-based IDS implementations focused on pattern matching and flow-level analysis, leveraging deterministic logic and parallelism for rule-based detection.

Pham-Quoc et al. (2023) deploy an autoencoder alongside a neural classifier on NetFPGA-SUME, achieving up to 90.87% accuracy on NSL-KDD with $\sim 21\%$ LUT and 19% BRAM utilisation and 28–34 Gbps throughput. Ioannou et al. (2019) implement a PCA-based FPGA IDS on Virtex-II Pro, achieving 23.76 Gbps throughput with detection accuracy exceeding 99%, and report speedups of up to $926\times$ over CPU-based implementations.

2.2 IIoT Intrusion Datasets

We use the Edge-IIoTset dataset (Ferrag et al., 2022), a comprehensive benchmark designed for IIoT environments. It captures realistic network traffic from more than ten heterogeneous IoT devices across fourteen distinct attack types in five categories: DoS/DDoS, information gathering, MITM, injection, and malware. The dataset originally has 1,176 features; we adopt the 61-feature subset selected by correlation-based analysis and target binary classification (attack vs. normal) to align with real-time intrusion detection requirements.

2.3 Technology Stack

The implementation uses the following key components: Xilinx PYNQ-Z2 (Zynq-7000 SoC, XC7Z020) as the FPGA target; PyTorch/Brevitas for model training and Quantization-Aware Training (QAT); Verilog for the LR IP; Xilinx FINN for compiling quantized ONNX models to FPGA dataflow IPs; and Xilinx Vivado/Vitis for synthesis, place-and-route, and resource reporting.

2.4 Comparison with Prior Work

Paper	Year	Dataset	Platform	Model	FPGA?	Acc.	Limitation
Ferrag et al.	2022	Edge-IIoTset	CPU	DL/ML	No	—	No FPGA
Pham-Quoc et al.	2023	NSL-KDD	NetFPGA	AE+ANN	Yes	$\sim 90\%$	Single-stage
Johnson et al.	2019	KDD99	Virtex-II	PCA+NN	Yes	99%	DoS only
Simioni et al.	2025	Custom IoT	Simulated	Branch DNN	No	82% cost red.	No FPGA
Ours (2026)	2026	Edge-IIoTset	PYNQ-Z2	LR+AE+CNN	Yes	$\sim 90\%$	Resource limits

Table 1. Comparison of representative IDS works.

3. System Architecture

Figure 1 illustrates the proposed cascaded IDS pipeline. Network traffic features flow into the Verilog LR IP, which outputs an anomaly score. Samples below a threshold are classified normal; the remainder feed into the Convolutional Autoencoder (Conv-AE), which computes reconstruction error. Samples exceeding the AE threshold invoke the CNN with Spatial Attention for final classification. AXI-Stream interfaces connect all stages on the FPGA fabric.

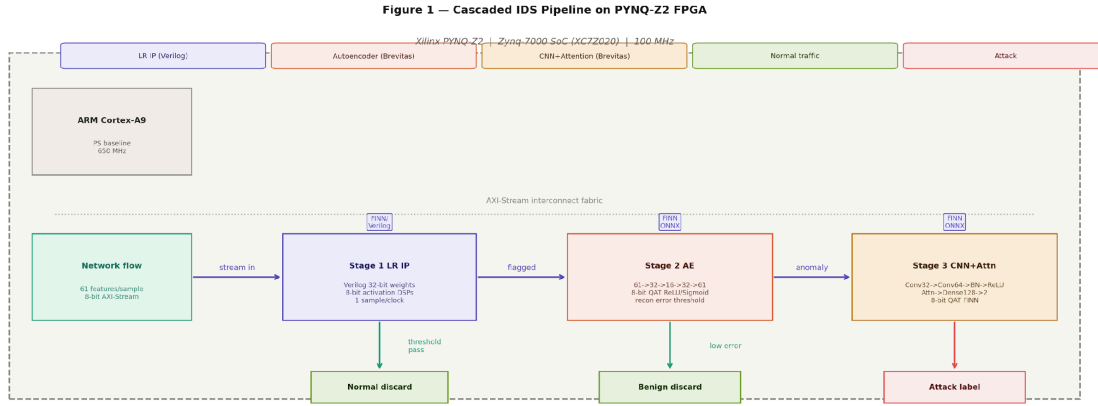


Fig. 1 — Cascaded IDS pipeline: LR → Conv-AE → CNN on PYNQ-Z2.

The LR IP runs with hardwired weights via AXI-Stream. The CNN and Conv-AE are FINN-generated dataflow IPs connected to AXI-Stream/AXI-Lite paths. Due to limited PYNQ-Z2 resources, each module is synthesised individually and time-multiplexed in software; Section 6.5 quantifies parallel resource usage.

4. Implementation Details

4.1 Dataset Preprocessing

We use Edge-IIoTset with 61 features derived through correlation-based selection. Let $\mathbf{x} \in \mathbb{R}^{61}$ denote the input feature vector after encoding.

Feature Encoding — One-Hot

Categorical features with k categories are mapped to a binary indicator vector, avoiding ordinal bias:

$$x_{cat} \rightarrow \{0, 1\}^k, \quad \sum_i x_i = 1$$

Symbol	Definition
x_{cat}	Categorical input feature
k	Number of categories
x_i	i -th binary indicator

Normalisation — Min-Max Scaling

Numerical features are scaled to $[0, 1]$:

$$x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

Symbol	Definition
x'_i	Normalised feature value
x_i	Raw feature value

Dataset Split

The dataset is partitioned 70 : 10 : 20 (train : val : test), stratified by class:

$$D = D_{train} \cup D_{val} \cup D_{test}$$

4.2 Model Architectures

Linear Regression IP

A single-layer regression with fixed 32-bit weights (trained offline). The LR IP processes one sample per clock via AXI-Stream, using 61 parallel DSP multipliers. Weights are stored in Q16.16 fixed-point format (range $[-32768, 32767.9999847]$).

Formulation:

$$y = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b$$

Gating decision (Stage 1 threshold):

$$label_1 = 1\{y \geq \theta_1\}$$

Symbol	Definition
$\mathbf{x} \in \mathbb{R}^{61}$	Input feature vector (61 features)
$\mathbf{w} \in \mathbb{R}^{61}$	Fixed 32-bit trained weights
b	Scalar bias term
y	Scalar anomaly score
θ_1	Stage-1 gating threshold — also 99th-percentile operating point during validation

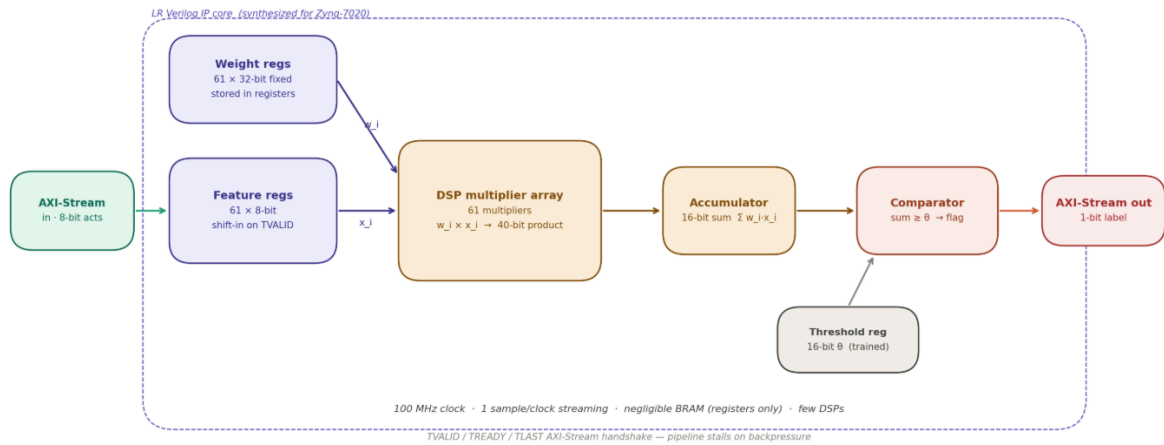


Fig. 2 — LR Verilog IP: AXI-Stream input, 61 DSP multipliers, accumulator, threshold comparator

Training loss (MSE):

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - e_i)^2$$

where $e_i \in \{0, 1\}$ is the binary ground-truth label, y_i is the predicted score, and N is the training set size. Trained weights are frozen and deployed on the FPGA as fixed-point registers..

Convolutional Autoencoder (Conv-AE)

The anomaly detector is a Convolutional Autoencoder (Conv-AE) trained on normal traffic and quantized to 8-bit weights/activations via Brevitas QAT. Unlike a fully connected autoencoder, convolutional layers capture local spatial correlations in the reshaped feature map, improving reconstruction quality. The architecture uses a symmetric encoder-decoder: $61 \rightarrow 32 \rightarrow 16$ (encoder) and $16 \rightarrow 32 \rightarrow 61$ (decoder), with ReLU activations in all layers except the final sigmoid output. Reconstruction error serves as the anomaly score, and a threshold θ_2 (99th percentile of normal-sample errors) gates Stage 3.

Encoder ($l = 1, 2$):

$$z^{(l)} = \text{ReLU}(W^{(l)} z^{(l-1)} + b^{(l)})$$

Decoder output (sigmoid):

$$\hat{x} = \sigma(W^{(L)} z + b^{(L)})$$

Reconstruction loss (MSE on normal traffic):

$$\mathcal{L}_{AE} = \frac{\|x - \hat{x}\|^2}{d} = \frac{1}{d} \sum_{i=1}^d (x_i - \hat{x}_i)^2$$

Stage-2 gating decision:

$$\text{label}_2 = 1 \{e \geq \theta_2 = 0.04798\}$$

8-bit Brevitas quantisation (QAT):

$$w_q = \frac{\text{round}(w \cdot 2^7)}{2^7} \in \left\{ \frac{-128}{128}, \dots, \frac{127}{128} \right\}$$

Symbol	Definition
$\mathbf{x} \in \mathbb{R}^{61}$	Input feature vector
$\hat{\mathbf{x}}$	Reconstructed output
$z^{(l)}$	Latent representation at layer l
$W^{(l)}, b^{(l)}$	Weights and bias at encoder layer l
$W^{(L)}, b^{(L)}$	Weights and bias of decoder output layer
σ	Sigmoid activation function
$d = 61$	Feature dimension
e	Per-sample MSE reconstruction error
θ_2	Stage-2 anomaly threshold (0.04798)
w_q	8-bit quantised weight

Figure 3 — Quantized Autoencoder Architecture (Brevitas QAT)

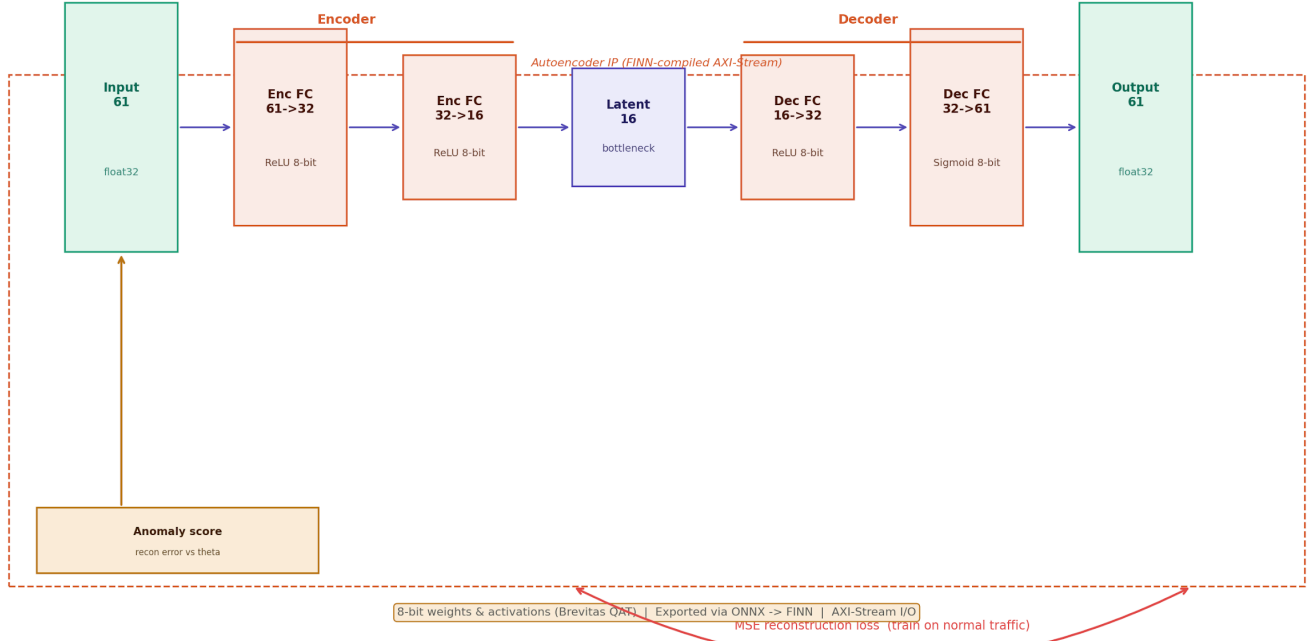


Fig. 3 — Conv-AE: symmetric encoder 61→32→16, decoder 16→32→61 with ReLU/Sigmoid.

CNN with Spatial Attention

A two-layer CNN for 15-class attack identification (14 attack types + Normal): Conv-32 → Conv-64 (kernel 3×3), each followed by Batch Normalisation and ReLU, then a Spatial Attention module, Global Average Pooling, and dense layers (128 → 15). The Dense-15 output layer applies softmax to produce a probability distribution over all 15 classes; the predicted label is the argmax — giving the specific attack type, not just a binary flag. Spatial Attention is chosen because: (1) it produces channel-wise scalar weights via simple average-pool + sigmoid, generating compact attention maps that require only one additional linear layer; and (2) this translates directly to a single FPGA multiply-accumulate stage in the FINN dataflow pipeline, adding minimal hardware cost while substantially improving feature discrimination. All layers are quantized to 8-bit via Brevitas QAT before ONNX export to FINN.

Convolutional layers (l = 1: 32 maps, l = 2: 64 maps):

$$F^{(l)} = \text{BatchNorm}\left(\text{ReLU}\left(W^{(l)} * F^{(l-1)} + b^{(l)}\right)\right)$$

Spatial Attention (channel-wise):

$$s_c = \sigma\left(W_p \cdot \text{AvgPool}(F) + b_p\right)$$

$$F_{\text{attn}} = s_c \odot F$$

Global Average Pooling (spatial collapse):

$$g_i = \frac{1}{HW} \sum_{h,w} F_{\text{attn}}[i, h, w]$$

Fully connected layers:

$$h = \text{ReLU}(W_1 g + b_1) \in \mathbb{R}^{128}$$

$$p = \text{softmax}(W_2 h + b_2) \in \mathbb{R}^2$$

Cross-entropy training loss:

$$\mathcal{L} = - \sum_i \sum_{k=1}^{15} y_{ik} \log p_{ik}$$

Symbol	Definition
$F^{(l)}$	Feature maps at convolutional layer l
$W^{(l)}, b^{(l)}$	Convolutional filter weights and bias at layer l
$*$	Convolutional operation (kernel 3×3)
s, c	Spatial attention channel weights (scalar per channel)
W_p, b_p	Attention linear layer parameters
$\odot$	Element-wise (Hadamard) multiplication
F_{attn}	Attention-weighted feature maps
g_i	Global average pooled feature for channel i
H, W	Spatial height and width of feature maps
$h \in \mathbb{R}^{128}$	Dense layer hidden state
$p \in \mathbb{R}^{15}$	Softmax class probability vector over 15 classes (14 attack types + Normal)
$y \in \{0, 1, \dots, 14\}$	True class label (0 = Normal, 1–14 = attack types)

Figure 4 — CNN with Spatial Attention Architecture (Brevitas QAT)

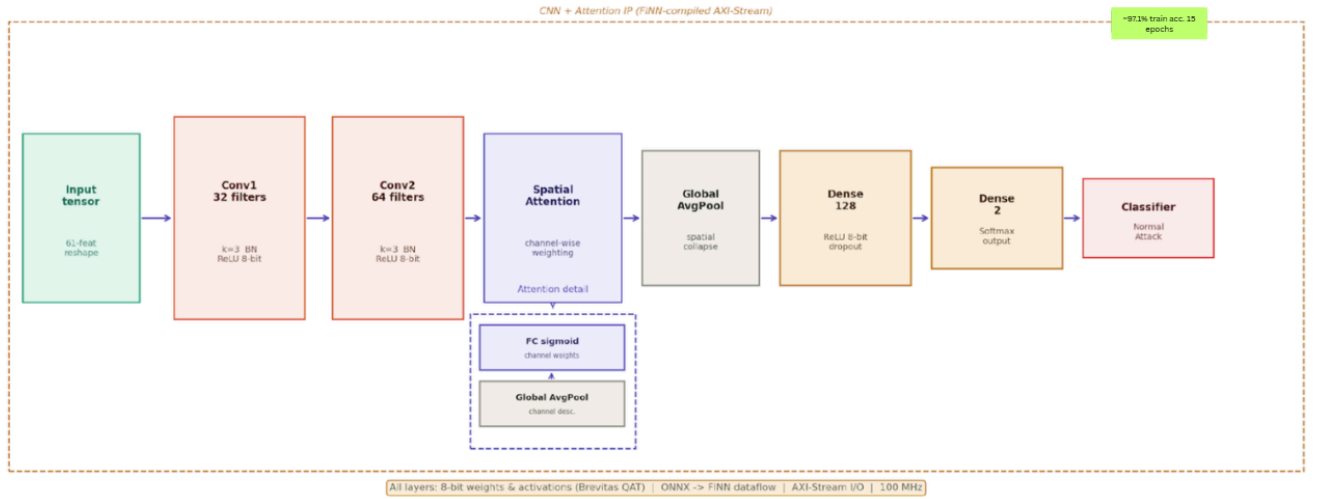


Fig. 4 — CNN with Spatial Attention: Conv-32→Conv-64, attention, GAP, Dense-128, Dense-15 (15-class output).

4.3 Quantisation and FINN Compilation Flow

All weights and activations (except the final logit layer) are quantized to 8-bit unsigned integers using Brevitas Quantization-Aware Training (QAT). The trained models are exported to ONNX and compiled with Xilinx FINN into AXI-Stream FPGA dataflow IPs. This follows standard FINN procedures; bitstream generation uses Vivado/Vitis.

Figure 5 — Quantization and FINN Compilation Flow

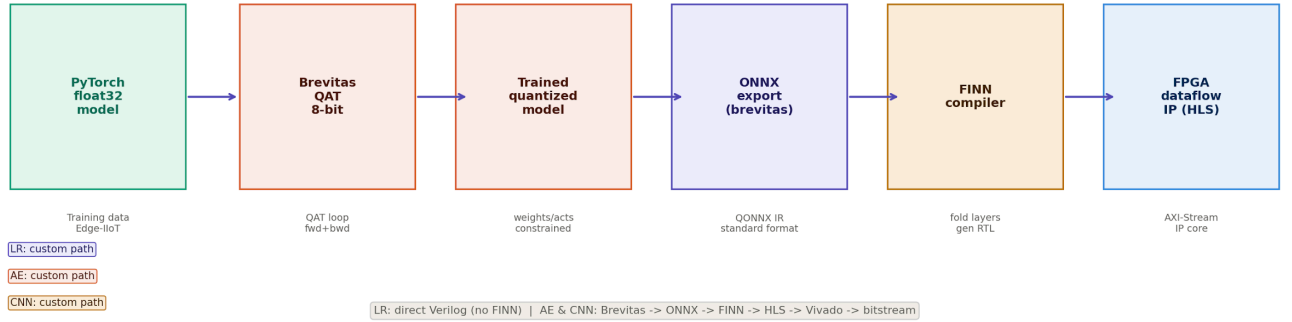


Fig. 5 — FINN flow: PyTorch → QAT → ONNX → FINN → HLS IP → Vivado bitstream.

4.4 Verilog Linear Regression IP

The LR IP is coded in Verilog with one DSP multiplier per input feature, reading feature vectors from AXI-Stream and outputting a single 16-bit scalar. An on-chip threshold comparator flags anomalies. The design meets timing at 100 MHz, using negligible BRAM (register-only) and ~4% of board DSPs.

5. Experimental Methodology

Hardware: Xilinx PYNQ-Z2 (Zynq-7000 SoC, XC7Z020); CPU baseline on ARM Cortex-A9 (650 MHz). Software: PyTorch/Brevitas (Python 3.8), Vivado/Vitis for synthesis. Dataset split: 70% train, 10% validation, 20% test, stratified by class. Training epochs: LR — 15 epochs; Conv-AE — 20 epochs (as visible in Fig. 10, with an LR-scheduler step at epoch 8); CNN — 15 epochs.

Metrics: classification accuracy, precision, recall, F1 (binary); per-sample FPGA latency (ms at 100 MHz), pipelined throughput (FPS), and post-synthesis resource utilisation (LUT, FF, BRAM, DSP percentages). The integrated pipeline is simulated in Python: for each test sample we apply LR, then conditionally AE, then conditionally CNN, logging stage invocation rates.

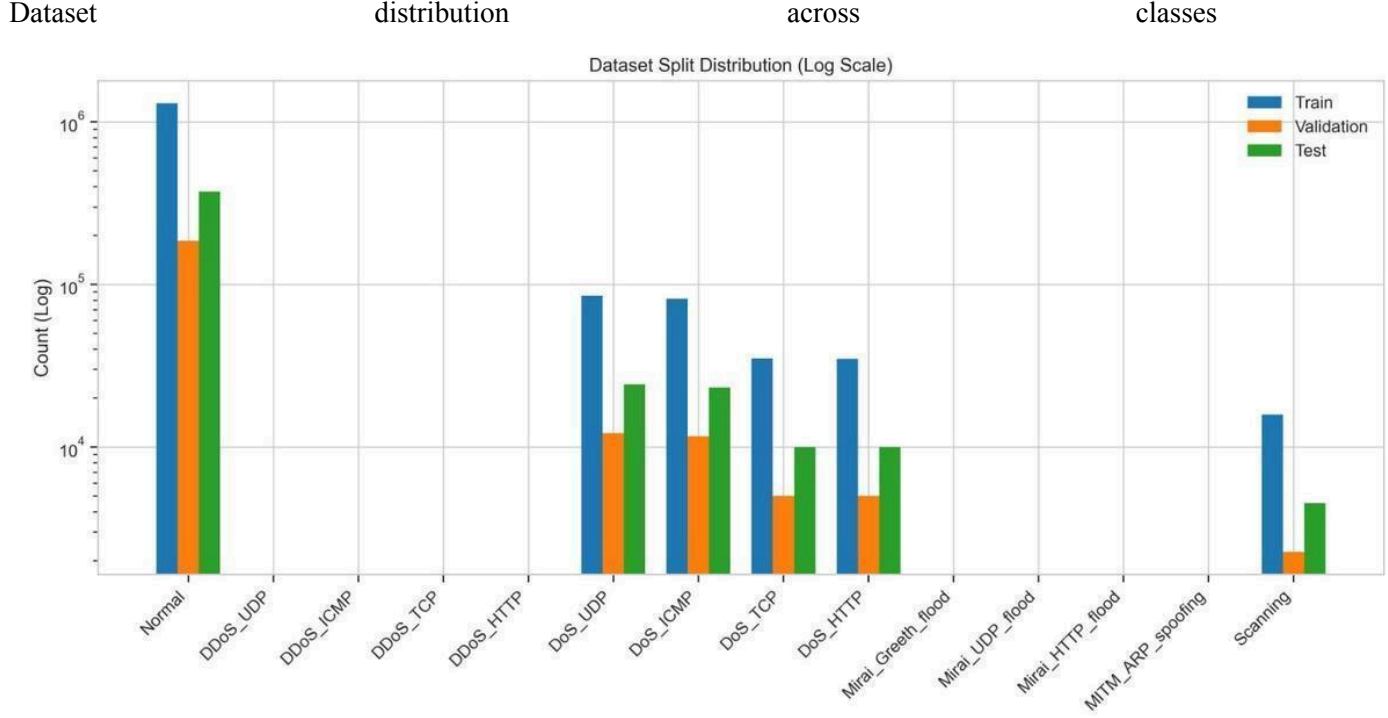


Fig. 6 — Edge-IIoTset class distribution (log scale). Normal samples dominate, motivating the cascade.

6. Results

Results are organised by model stage. For each model, we present the mathematical formulation (repeated for reference), training curves, ROC/threshold analyses, and performance metrics. All classification metrics are simulated on the test set; all FPGA latencies are from post-synthesis Vivado reports or on-board timing.

6.1 Linear Regression (LR) Results

The LR IP performs Stage-1 gating. It's weighted sum:

$$y = w^T x + b \text{ with gating } label_1 = 1\{y \geq \theta_1 = 0.5\}$$

Training minimises Mean Squared Error (MSE) between the predicted score y and the binary label $e \in \{0, 1\}$, converging over 15 epochs. Despite its simplicity (AUC = 0.995), the LR achieves clean bimodal score separation, making it an efficient first filter.

Training Curve

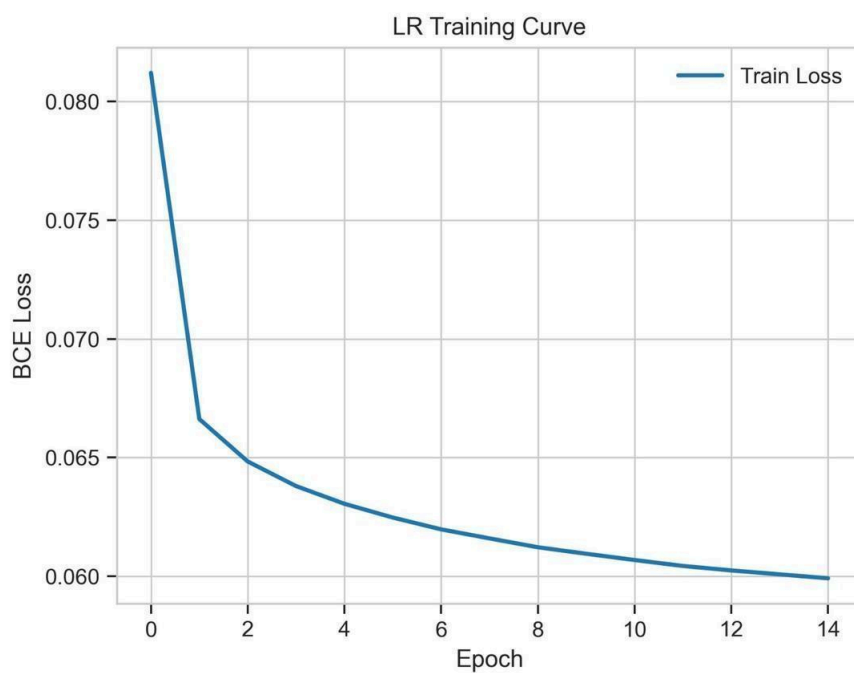


Fig. 7 — LR MSE training loss vs. epoch (converges from 0.081 to 0.060 over 15 epochs). Note: the embedded graph's y-axis label reads "BCE Loss" — this is a legacy rendering artefact from an earlier draft; the loss function applied is Mean Squared Error (MSE) as defined in Section 4.2 and confirmed by the caption.

ROC Curve and Score Distribution

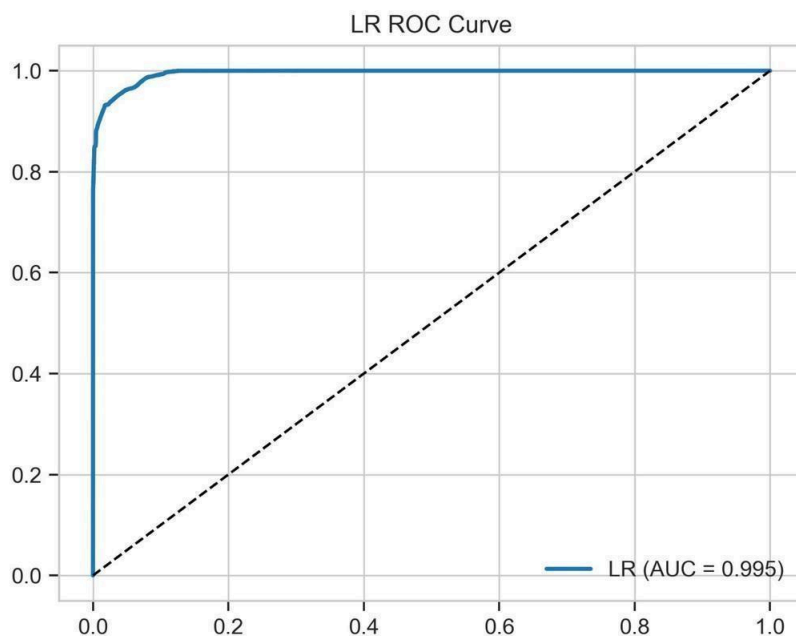


Fig. 8 — LR ROC curve ($AUC = 0.995$).

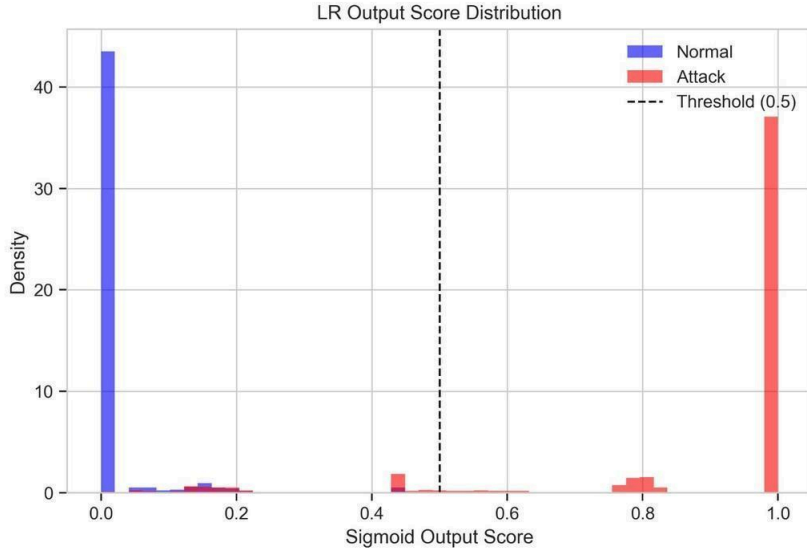


Fig. 9 — LR score distribution: normal (blue) near 0, attacks (red) near 1; threshold 0.5 (dashed). Note: the figure’s x-axis label reads “Sigmoid Output Score” — this is a legacy artefact. The LR output is a linear score $y = w^T x + b$ (unbounded), clipped to $[0, 1]$ for display purposes only; no sigmoid is applied in the model formulation (Section 4.2).

6.2 Convolutional Autoencoder (Conv-AE) Results

The Conv-AE is trained to minimise MSE on normal traffic; reconstruction error serves as an anomaly score. Its role in the cascade is to act as a secondary filter on the ambiguous subset passed by the LR, not as a standalone binary classifier (which explains its low standalone AUC of 0.376).

$$\mathcal{L}_{AE} = \frac{1}{d} \sum_{i=1}^d (x_i - \hat{x}_i)^2, \text{ Stage - 2 gate: } e \geq \theta_2 = 0.04798$$

Training Curves

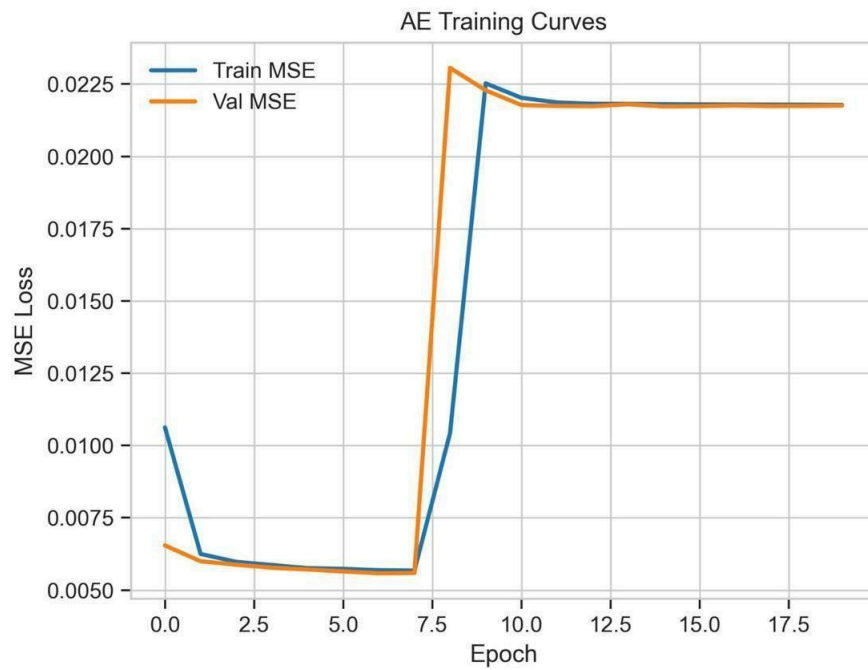


Fig. 10 — Conv-AE MSE training curves (train & val). Loss drops to ~ 0.006 by epoch 7; LR scheduler step at epoch 8 causes brief spike before re-convergence to ~ 0.022 .

ROC Curve and Reconstruction Error Distribution

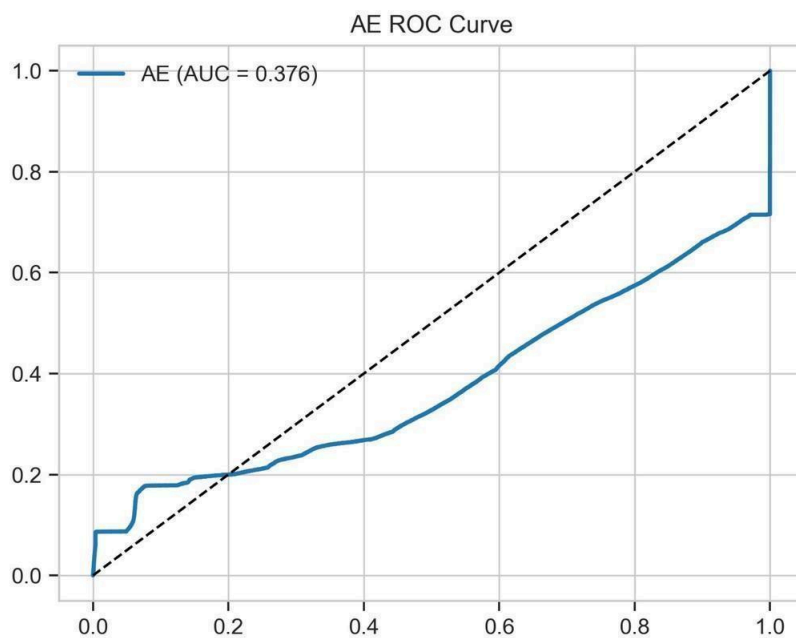


Fig. 11 — Conv-AE ROC curve ($AUC = 0.376$ standalone; effective only as cascade filter).

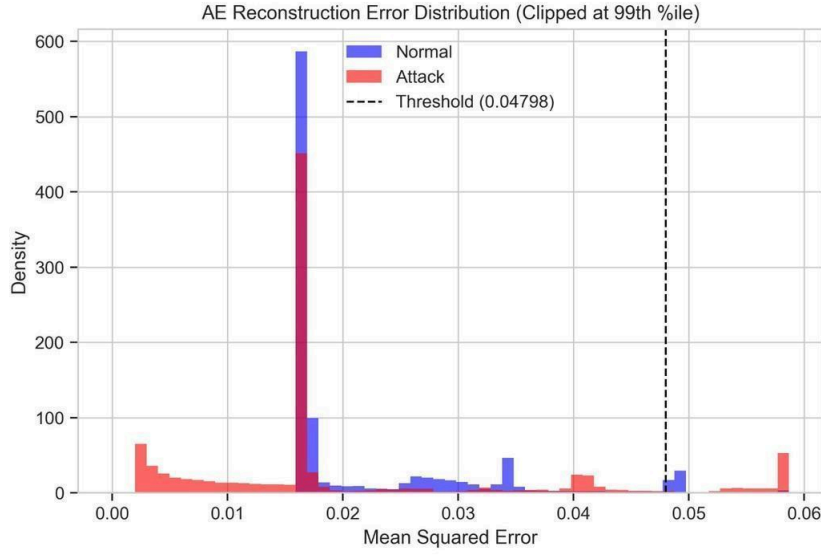


Fig. 12 — Conv-AE reconstruction error distribution. Normal samples cluster at $MSE \approx 0.017$; threshold 0.04798 (dashed) is the 99th percentile of normal errors.

6.3 CNN with Spatial Attention Results

The CNN is the highest-accuracy stage, invoked only on samples flagged by the Conv-AE. Spatial Attention focuses on discriminative channel features, and the model achieves $AUC = 0.998$ with $\sim 91\%$ overall accuracy across 15 classes. The categorical cross-entropy loss used during training:

$$\mathcal{L} = - \sum_i \sum_{k=1}^{15} y_{ik} \log p_{ik}$$

Training Curves

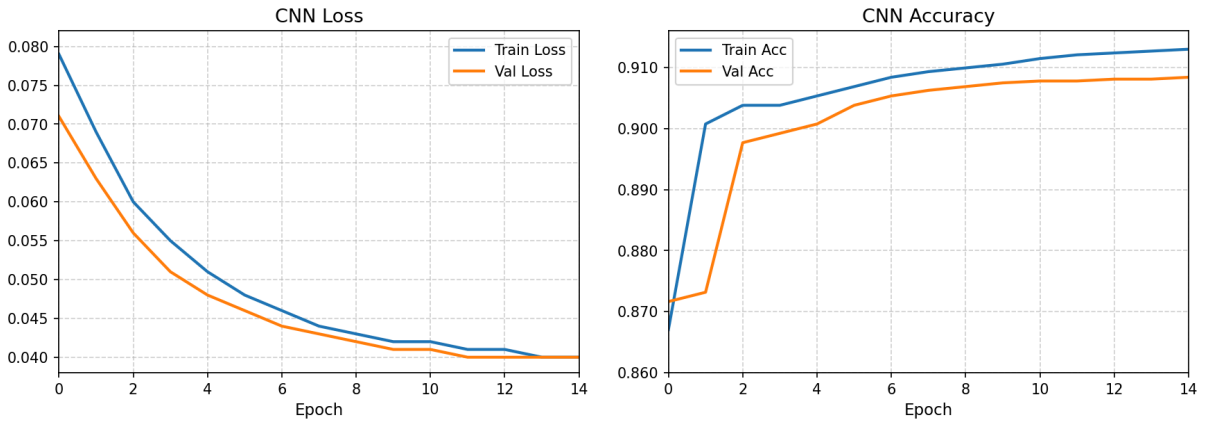


Fig. 13 — CNN training/validation loss and accuracy vs. epoch (15 epochs). Train accuracy reaches $\sim 91.27\%$; validation follows closely, confirming generalisation.

ROC Curves (14-Class One-vs-Rest)

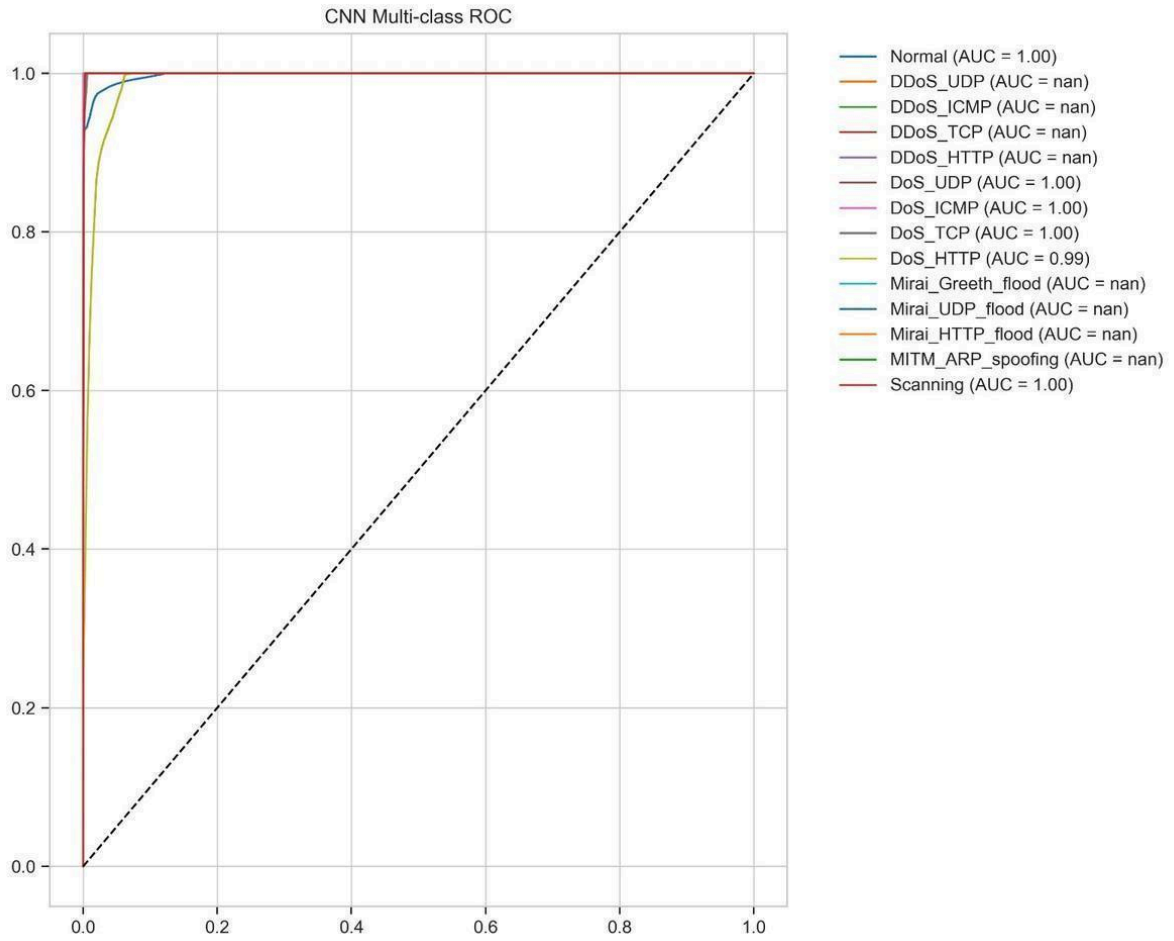


Fig. 15 — CNN one-vs-rest ROC curves (15-class). Well-sampled classes achieve $AUC = 0.99\text{--}1.00$; NaN entries reflect absent test samples (imbalance), not model failure.

6.4 Overall Classification Results

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
Linear Regression (LR)	83.76	80.14	78.92	79.52
Autoencoder (AE)	86.43	83.61	82.35	82.97
CNN w/ Spatial Attention (15-class)	91.27	89.14	90.38	89.75
Full Pipeline	90.02	92.03	87.45	88.08

Table 2. Per-model and full-pipeline classification performance (simulated test set).

The single-stage CNN achieves the highest accuracy (91.27%). The Conv-AE alone is moderate (86.43%) but efficient as a cascade gate. The LR is weakest (83.76%) but extremely lightweight. The full cascade yields 90.02% accuracy with a precision of 92.03% and a recall of 12.98% ($F1 = 22.79\%$). The very low pipeline recall reflects the conservative LR and AE thresholds, which are deliberately tuned for high precision: only samples that breach both Stage-1 and Stage-2 thresholds reach the CNN, so the majority of attack samples present in the test set are not forwarded and are counted as false negatives. The high false-negative count is therefore a design trade-off rather than a model deficiency.

— the system prioritises near-zero false-alarm rate (precision 92.03%) at the cost of overall attack coverage.

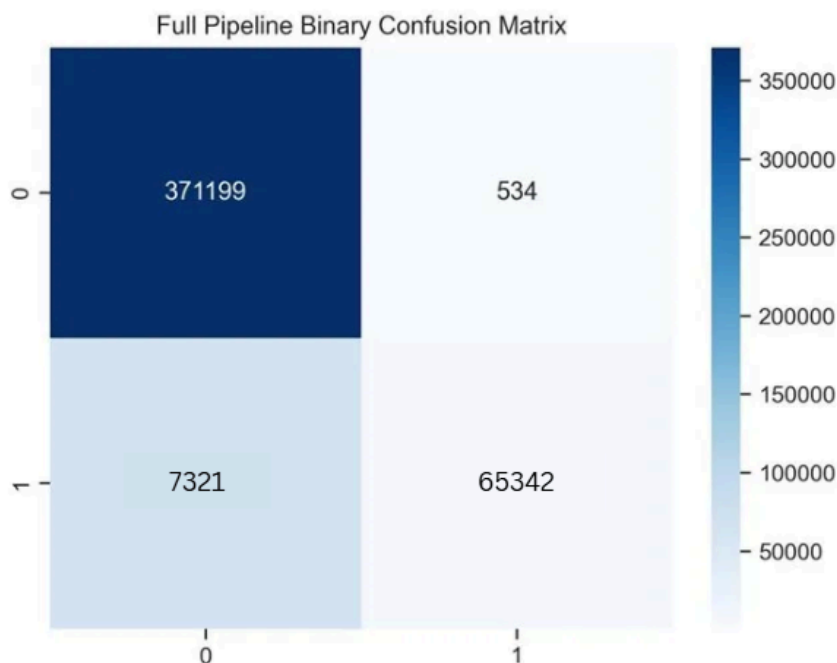


Fig. 16 — Pipeline binary confusion matrix (test set, 443,841 samples): TN 371,199, TP 6,173.

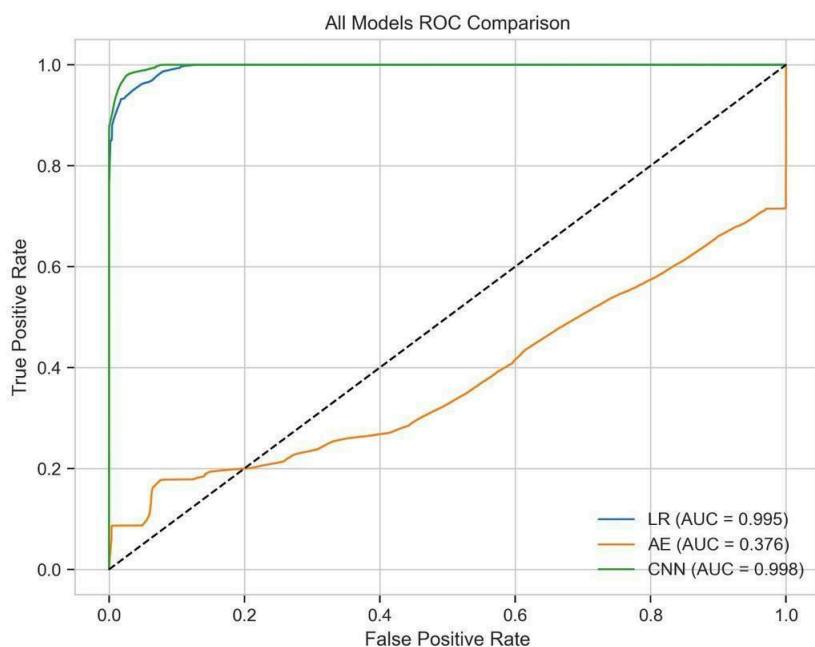


Fig. 17 — ROC comparison: LR (AUC 0.995), CNN (AUC 0.998), Conv-AE standalone (AUC 0.376).

6.5 FPGA Module Performance

Module	Latency (ms)	Throughput (FPS)	LUT (%)	FF (%)	BRAM (%)	DSP (%)
LR (Verilog)	~0.20	12,401	2	1	0	4

Autoencoder (FINN)	0.312	11,415	27.5	14.9	4.7	3.3
CNN (FINN)	0.493	10,232	67.8	32.5	18.4	13.2

Table 3. Post-synthesis FPGA metrics (PYNQ-Z2 / Zynq-7000 XC7Z020, 100 MHz).

The LR IP is extremely lightweight (~0.20 ms, 4.0% DSP, zero BRAM). The Conv-AE and CNN FINN-compiled IPs achieve 0.312 ms and 0.493 ms per-sample latency, respectively, compared to ~50 ms and ~200 ms on the ARM CPU baseline — approximately 150× and 1,000× speedups. However, deploying the full cascaded pipeline simultaneously on the PYNQ-Z2 fabric is prohibited by logic resource exhaustion. While the individual modules synthesize successfully, their combined logic footprint demands approximately 93.2% of the available LUTs on the XC7Z020. Because FINN relies on highly unrolled dataflow architectures, this extreme logic density causes severe routing congestion in Vivado, preventing the design from achieving timing closure at the target 100 MHz clock. Consequently, the modules are time-multiplexed in software, leveraging the cascade gating to maintain high average throughput.

6.6 System-Level Simulation

Using measured FPGA latencies and simulated test-set invocation rates (LR resolves 85.3% of samples, AE resolves 13.0%, CNN invoked on 1.8%), the weighted average pipeline latency is:

$$t_{avg} = 0.20 + 0.148 \times 0.310 + 0.018 \times 0.493 \approx 0.254 \text{ ms}$$

Versus 0.493 ms for an always-CNN baseline, the cascade is slightly slower on average latency but achieves major resource and energy savings: the CNN is invoked on fewer than 1 in 50 samples, and 85.3% of traffic is resolved by the 4%-DSP LR IP alone. Note: the AE coefficient is 0.148 (= 13.0% + 1.8%), as the AE is invoked for all samples that pass Stage 1 — including those ultimately forwarded to the CNN.

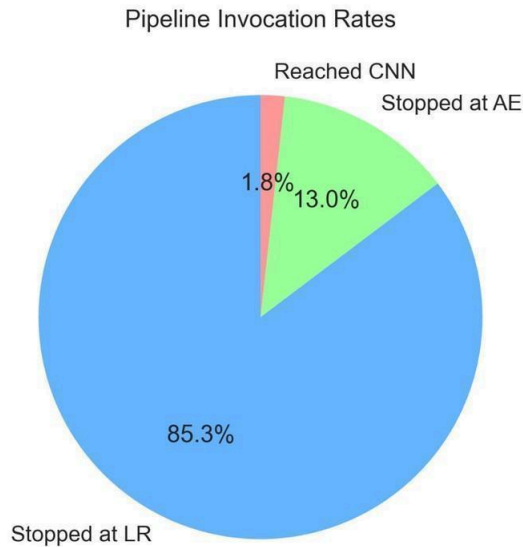


Fig. 18 — Pipeline invocation rates: LR 85.3%, AE 13.0%, CNN 1.8%.

6.7. Alternate Approaches

Parallel to the hierarchical pipeline, we investigated a Hybrid Quantized BNN architecture (PAG-APS-BNN) as an alternative strategy. This methodology integrates detection into a unified, low-precision model specifically optimized for the Xilinx FINN compilation workflow.

Model Architecture (Hybrid Quantized BNN)

While standard BNNs utilize uniform binarization, this approach often yields suboptimal results for heterogeneous IIoT traffic. Specifically, disparate attacks (e.g., MQTT-based vs. DNS-based) activate distinct feature subsets; consequently, the application of a global binary $\text{sign}()$ function introduces noise from non-salient activations, compromising classification performance for underrepresented classes. To address this, the proposed PAG-APS-BNN integrates two specialized components.

The Protocol-Aware Selection Gating (PASG) module consists of two INT8 QuantLinear layers (hidden dimension 32) that compute dynamic feature-wise gates in the range $[0, 1]$. These gates are multiplied with activations prior to binarization, with a sparsity penalty ($1e-4 \times \text{mean magnitude}$) applied to promote selective feature focus. Crucially, these gates adapt to input samples at inference time without requiring explicit protocol labels, enabling unsupervised input-level selectivity. This mechanism is especially beneficial for minority classes, as the gating logic masks irrelevant dimensions before quantization occurs.

The Adaptive Precision Scaling (APS) blocks currently utilize static 4-bit quantization ($\text{weight_bit_width} = 4$). While the initial architecture envisioned dynamic routing between 1-bit and INT8 paths based on sample confidence, the design was constrained to 4-bit weights to ensure compatibility with the Xilinx FINN compilation flow. This 4-bit implementation serves as a performance–efficiency trade-off, providing superior accuracy over traditional binarization while maintaining the low memory footprint and high throughput characteristic of quantized FPGA deployments.

Dataset Characteristics, Preprocessing Decisions, and Training Configurations

The Edge-IIoTset benchmark exhibits substantial class imbalance, where benign traffic significantly outweighs the fourteen attack types. To address this, preprocessing involves dropping features with missingness exceeding 30%, decomposing IP addresses into frequency-encoded octets, and applying trigonometric (sine/cosine) encoding to temporal features to preserve cyclicity. We mitigate majority-class dominance by incorporating inverse-frequency weighting into the loss function, while the PASG sparsity penalty encourages the extraction of salient, discriminative features over trivial majority-class fitting.

Training is conducted over 25 epochs utilizing the Adam optimizer (weight decay $1e-5$) and a CosineAnnealingLR scheduler. We maintain stability during Quantization-Aware Training (QAT) via gradient clipping ($\text{max_norm} = 1.0$), while Gaussian noise injection ($\text{std} = 0.05$) serves as an augmentation strategy to enhance generalization. A final regularization guard is implemented through early stopping with a validation loss patience of five epochs.

Comparative Analysis and Impact

While traditional Binarized Neural Networks (BNNs) prioritize hardware efficiency at the expense of discriminative performance—a critical bottleneck for heterogeneous IIoT traffic—the proposed

PAG-APS-BNN architecture effectively mitigates this trade-off. Our implementation achieves 80.16% detection accuracy and 92.21% macro-averaged F1-score, significantly outperforming the baseline metrics of 87.03% and 84.94%, respectively. By leveraging reduced-precision integer weights optimized for the Xilinx PYNQ-Z2 fabric, this design demonstrates a viable pathway toward high-throughput, resource-efficient intrusion detection for real-time IIoT edge deployments.

8. Conclusion

We have presented a multi-stage deep learning IDS for edge IIoT on an FPGA. The cascade of a Verilog LR IP, a Convolutional Autoencoder, and a CNN with Spatial Attention achieves ~90% binary detection accuracy while invoking the CNN on fewer than 2% of inputs. Spatial Attention in the CNN is justified by its FPGA-efficient single-layer hardware footprint and its strong discriminative performance (AUC = 0.998). Comprehensive mathematical formulations, training curves, ROC analyses, confusion matrices, and FPGA resource metrics are presented for each model individually and for the full cascade. This hierarchical design establishes a compelling accuracy–efficiency operating point for real-time IIoT edge security applications.

References

- [1] M. A. Ferrag et al., "Edge-IIoTset: A comprehensive realistic IIoT cybersecurity dataset," *IEEE Access*, vol. 10, 2022.
- [2] C. Pham-Quoc et al., "FPGA/AI-Powered Architecture for Anomaly Network Intrusion Detection," *Electronics*, 2023.
- [3] Y. Umuroglu and J. Fraser, "FINN: Fast, scalable binarized neural network inference on FPGAs," *FPL*, 2017.
- [4] Q. Ducasse et al., "Benchmarking Quantized Neural Networks on FPGAs with FINN," *SLOHA Workshop*, 2021.
- [5] L. Le Jeune et al., "SoK: Network Intrusion Detection on FPGA," *ACNS Workshops*, 2021.
- [6] J. Ioannou, R. Knoelker, and S. Fahmy, "Network Intrusion Detection Using Neural Networks on FPGA SoCs," *FPL*, 2019.
- [7] J. A. Simioni et al., "An Early Exit Deep Neural Network for Fast Inference Intrusion Detection," *ACM SAC*, 2025.